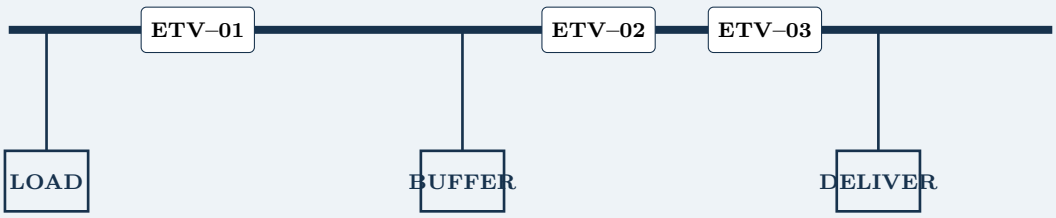
MED–ETV / ENGINEERING NOTE

First Iteration Simulation Prototype

Three overhead ETVs, vertical pod transfer, and a plain-text live monitor

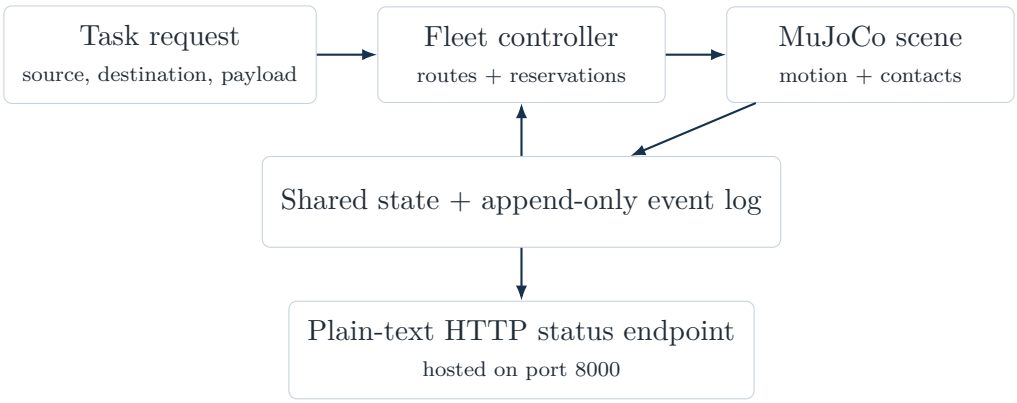


Concept illustration only — not to scale

Purpose	Define a small, testable first iteration with minimal live observability.
Simulation	MuJoCo scene with Python orchestration and a logical rail graph.
Initial fleet	Three simplified ETV bodies; one shared overhead rail with downward vertical guide-rail branches; stations sit below and off the main route.
Scope boundary	Simulation prototype only; no clinical deployment or certified safety claims.
Source basis	MED–ETV Simulation-First System Design Proposal / SEMP, Draft 1.0, October 2026.

1 First-iteration objective

Build a compact, repeatable demonstration of three ETVs that travel horizontally along an overhead rail, lower a carrier into a loading or delivery pod, complete a basic load/unload transaction, and return to rail height. Observability is provided by a small hosted plain-text status page and append-only live log; no dashboards or graphical control panels are included.



Implementation principle. Keep the first iteration in one Python process. Use MuJoCo for the scene and motion representation; use a separate logical graph for route and conflict reasoning. Do not add ROS 2, ZeroMQ, distributed agents, detailed carrier dynamics, or statistical experiments in this iteration.

2 Minimum system behaviour

1. Three ETVs exist in the scene and have unique IDs: ETV–01, ETV–02, and ETV–03.
2. Each ETV moves along the horizontal main rail to a branch junction and stops above the selected station; stations are off-route, below the main rail.
3. At the branch, a vertical guide rail forms a downward C-shaped transfer path to the station pod. The carrier lowers to the pod and rises back to rail height; horizontal travel is inhibited during transfer.
4. Loading and unloading update a payload/job state, rather than requiring detailed physical grasping.
5. A reservation controller protects the main-rail segment, branch junction, vertical guide rail, and station/pod zone as separate conflict resources.
6. The system records timestamped events and periodic position/state telemetry.

3 Motion and transfer concept

Represent each ETV with the minimum state needed by the controller and status monitor:

$$s_i = (x_i, z_i, v_i, q_i, j_i, p_i)$$

where x_i is rail position, z_i is vertical carrier position, v_i is horizontal speed, q_i is the operating state, j_i is the job ID, and p_i is the payload status. For the first iteration, acceleration and braking can be bounded and simplified; the model is not intended to predict a production vehicle.

4 Hosted plain-text live monitor

There are no dashboards, ETV renderings, charts, or graphical controls in this iteration. Expose a minimal read-only status page over HTTP on port `8000`. A browser displays plain, monospaced text

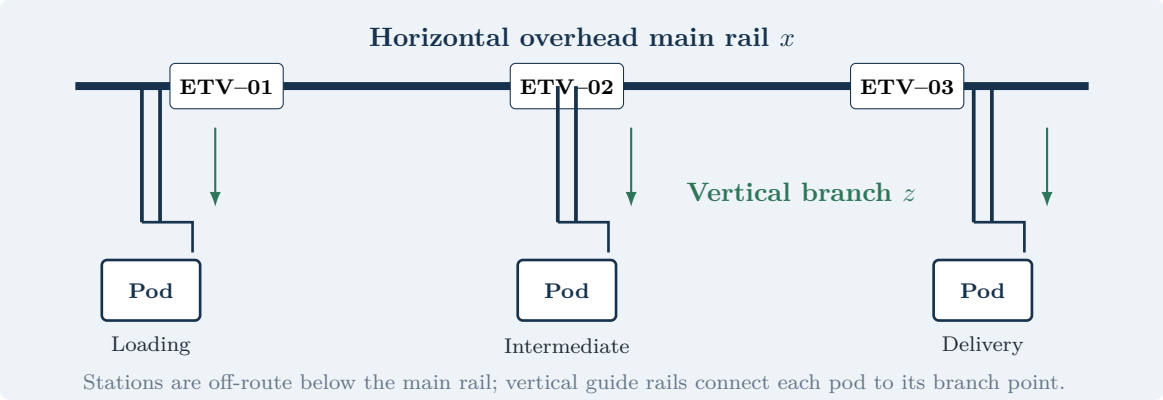


Figure 1: First-iteration topology: stations sit below and off the horizontal route. Each vertical guide-rail branch provides controlled carrier access to its pod; the C-shaped path is schematic.

refreshed at a fixed interval (for example, once per second). The simulation remains the source of truth; the page must not invent or independently store status values.

At the top, print one live status record for every station/pod and every ETV. Include the state and coordinates in a consistent format. Example values below are illustrative only:

```
MED-ETV LIVE STATUS | SIM-TIME 00:14:08 | SYSTEM RUNNING
POD-L01 | STATE=LOADING | ETV=ETV-01 | X: 2.40 m | Y: 0.00 m | Z: 0.18 m | P:
Carrier-016
POD-B01 | STATE=IDLE | ETV=NONE | X: 8.00 m | Y: 0.00 m | Z: 0.00 m | P: NONE
POD-D01 | STATE=UNLOADING | ETV=ETV-02 | X: 12.50 m | Y: 0.00 m | Z: 0.12 m | P:
Carrier-015
ETV-01 | STATE=LOWERING | JOB=J-0042 | X: 2.40 m | Y: 0.00 m | Z: 0.18 m | P:
Carrier-016
ETV-02 | STATE=UNLOADING | JOB=J-0043 | X: 12.50 m | Y: 0.00 m | Z: 0.12 m | P:
Carrier-015
ETV-03 | STATE=WAITING | JOB=NONE | X: 9.20 m | Y: 0.00 m | Z: 0.00 m | P: NONE
```

After the status records, print the complete live event stream in chronological order, newest entries at the bottom. Keep all events for the full run; do not silently discard older records. If the output becomes large, the page may show the latest lines while a separate plain-text log file retains the complete history.

```
[00:14:03] INFO ETV-01 ARRIVED location=POD-L01 job=J-0042
[00:14:04] INFO ETV-01 RESERVATION-GRANTED resource=BRANCH-L01
[00:14:05] INFO ETV-01 LOWERING z=0.18 job=J-0042
[00:14:06] INFO ETV-03 WAIT reason=SEGMENT-OCCUPIED resource=X02
[00:14:08] INFO ETV-02 UNLOADING payload=Carrier-015 job=J-0043
```

The monitor needs only three behaviours: serve the current status text, append timestamped events to a persistent run log, and refresh the displayed state automatically. No login, filters, charts, route visualisation, buttons, or manual control are required for this iteration. A simple Python HTTP server is sufficient; bind to port `8000` and make the host address configurable for local or LAN access.

5 State model and safety logic

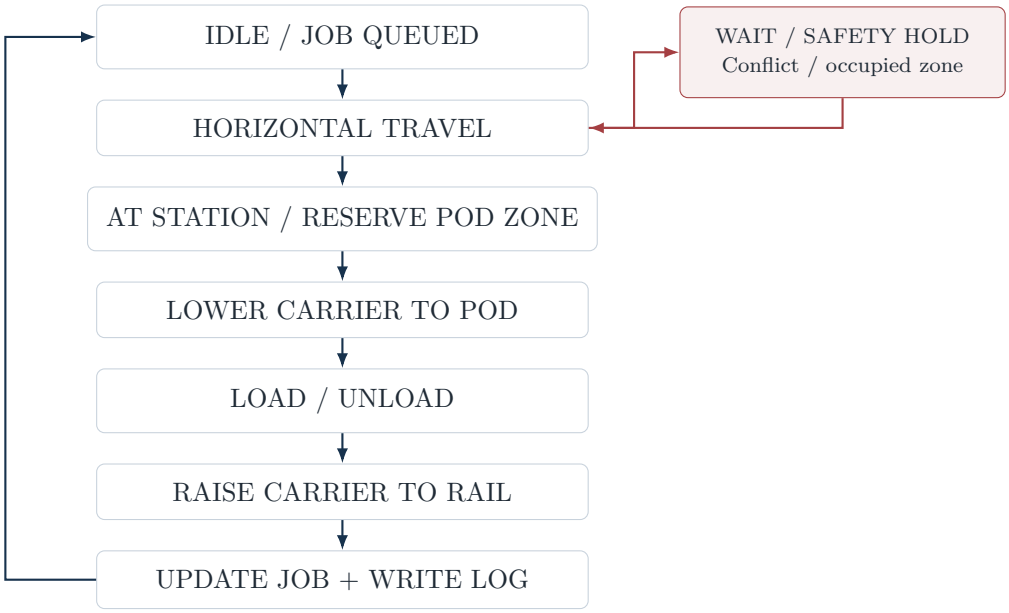


Figure 2: Simplified state machine. A blocked reservation causes a safe wait; no new route-planning or deadlock-recovery research is required for this first iteration.

Minimum collision-avoidance rule set

- Reserve a rail segment before entering it; release it after the vehicle clears it.
- Treat each lift column and pod service area as an exclusive resource.
- If a reservation is unavailable, stop before the protected boundary and log the hold.
- Do not permit horizontal movement while the carrier is below the rail-transfer limit.
- Add a geometric distance check as a secondary guard. MuJoCo collision contacts alone do not constitute a traffic-management policy.

6 Event model and logs

Use one shared event schema for the live status endpoint and exported CSV. Keep event logs append-only during a run.

Field	Meaning
`sim_time`	Simulation timestamp; use a monotonic time base.
`vehicle_id`	ETV–01, ETV–02, or ETV–03; blank for pod-level events where appropriate.
`job_id`	Job associated with the event, if any.
`event_type`	ARRIVED, RESERVED, LOWERED, LOADED, UNLOADED, RAISED, WAIT, DELIVERED, or ERROR.
`location`	Rail segment, loading pod, delivery pod, or transfer zone.
`status / detail`	Outcome and short explanation, including safety-hold reason.

Telemetry should record vehicle ID, x, z, horizontal speed, operating state, active job, payload state, reservation state, and safety-stop flag at a fixed interval. The plain-text endpoint may show recent lines; the append-only log file and CSV remain the complete run record.

7 Acceptance tests

ID	Test	Pass condition
T01	Fleet initialisation	Three unique ETVs and configured pods are visible.
T02	Horizontal tracking	Each ETV can travel to a commanded station and stop within its defined tolerance.
T03	Vertical transfer	A carrier lowers and raises; horizontal travel is inhibited during transfer.
T04	Payload transaction	Loading and unloading change the job and payload states correctly.
T05	Conflict handling	A denied reservation causes a stop/wait; no two ETVs occupy one exclusive resource.
T06	Live status consistency	Status page and event log reflect the same simulation state.
T07	Traceability	Required events and telemetry are exported to CSV with timestamps.
T08	Repeatability	A scripted scenario can be reset and replayed with the same configured initial conditions.

8 Build order and explicit exclusions

1. Build the rail, three ETV primitives, and loading/intermediate/delivery pod geometry in MuJoCo.
2. Implement horizontal target motion and bounded vertical transfer.
3. Add the state machine, payload/job state, resource reservations, and stop/wait logic.
4. Implement the shared event/telemetry model and CSV export.
5. Expose the shared state through a plain-text HTTP endpoint on port `8000`; append all events to a persistent log.
6. Run the acceptance tests and save a repeatable demonstration scenario.

Not part of iteration 1: realistic production geometry; full carrier swing dynamics; curved/branched rail networks; dynamic rerouting; decentralised negotiation; independent agent processes; ZeroMQ; ROS 2 integration; failure injection; Monte Carlo evaluation; hardware integration; clinical or certified safety claims.

9 Deliverable

A runnable MuJoCo/Python demonstration containing three ETVs, one simple overhead rail, at least one loading pod and one delivery pod (an intermediate station is optional but useful), vertical load/unload motion, reservation-based collision avoidance, a hosted plain-text live status page on port `8000`, and complete append-only logs. This is the smallest coherent prototype that demonstrates the transport transaction end-to-end while preserving the main MED–ETV architecture.

Prepared as a reduced first-iteration scope from the MED–ETV Simulation-First System Design Proposal / SEMP. Status and log examples are illustrative mock data; they define output format rather than measured results.